D8 Assignment

```
Reservoir_Porosity = float(input("What is the reservoir Porosity Value
? "))
Reservoir_Permeability = input(" What is the reservoir permeability
Value ? ")
Reservoir_Depth = input("What is the Depth of the reservoir in
meters ? ")
Reservoir_Net_Pay = input(" What is the Net Pay value of the reservoir
? ")
Reservoir_Water_Saturation = float(input(" What is the Reservoir Water
Saturation ? "))

Reservoir_Properties = {}

Reservoir_Properties["Reservoir_Porosity"] = Reservoir_Porosity
Reservoir_Properties["Reservoir_Permeability"] =
Reservoir_Permeability
Reservoir_Properties["Reservoir_Depth"] = Reservoir_Depth
Reservoir_Properties["Reservoir_Net_Pay"] = Reservoir_Net_Pay
Reservoir_Properties["Reservoir_Water_Saturation"] =
Reservoir_Water_Saturation

Reservoir_Properties["Reservoir_Hydrocarbon_Saturation"] = 1 -
Reservoir_Water_Saturation

Reservoir_Properties
```

```
{'Reservoir_Porosity': 0.22,
 'Reservoir_Permeability': '200',
 'Reservoir_Depth': '3000',
 'Reservoir_Net_Pay': '50',
 'Reservoir_Water_Saturation': 0.2,
 'Reservoir_Hydrocarbon_Saturation': 0.8}
```

```
oil_viscosity = input(" What is the Oil Viscosity Value ? ")
Gas_oil_ratio = input(" What is the Oil Gas Ratio ? ")
formation_water_salinity = input(" What is the formation Water
Salinity Value ? ")

Fluid_Properties = {}

Fluid_Properties['Oil_Viscosity'] = oil_viscosity
Fluid_Properties['Gas_Oil_Ratio'] = Gas_oil_ratio
Fluid_Properties["Formation_Water_Salinity"] =
formation_water_salinity

Fluid_Properties
```

```
{'Oil_Viscosity': '3.5',
 'Gas_Oil_Ratio': '800',
 'Formation_Water_Salinity': '20000'}
```

```
reservoir_performance = {
    "initial_pressure": 3500,      # Pressure in psi
    "current_pressure": 2500,      # Pressure in psi

}

All_Reservoir_Data = Reservoir_Properties

All_Reservoir_Data.update(Fluid_Properties)

All_Reservoir_Data.update(reservoir_performance)

All_Reservoir_Data
```

```
{'Reservoir_Porosity': 0.22,
 'Reservoir_Permeability': '200',
 'Reservoir_Depth': '3000',
 'Reservoir_Net_Pay': '50',
 'Reservoir_Water_Saturation': 0.2,
 'Reservoir_Hydrocarbon_Saturation': 0.8,
 'Oil_Viscosity': '3.5',
 'Gas_Oil_Ratio': '800',
 'Formation_Water_Salinity': '20000',
 'initial_pressure': 3500,
 'current_pressure': 2500}
```

```
All_Reservoir_Data.pop("Reservoir_Hydrocarbon_Saturation")

All_Reservoir_Data
```

```
{'Reservoir_Porosity': 0.22,
 'Reservoir_Permeability': '200',
 'Reservoir_Depth': '3000',
 'Reservoir_Net_Pay': '50',
 'Reservoir_Water_Saturation': 0.2,
 'Oil_Viscosity': '3.5',
 'Gas_Oil_Ratio': '800',
 'Formation_Water_Salinity': '20000',
 'initial_pressure': 3500,
 'current_pressure': 2500}
```

```
print("Reservoir Data Received!")

for key, value in All_Reservoir_Data.items():
    print(f"Parameter name: {key}, Parameter value: {value}")
```

```
Reservoir Data Received!
Parameter name: Reservoir_Porosity, Parameter value: 0.22
Parameter name: Reservoir_Permeability, Parameter value: 200
Parameter name: Reservoir_Depth, Parameter value: 3000
Parameter name: Reservoir_Net_Pay, Parameter value: 50
Parameter name: Reservoir_Water_Saturation, Parameter value: 0.2
Parameter name: Oil_Viscosity, Parameter value: 3.5
Parameter name: Gas_Oil_Ratio, Parameter value: 800
Parameter name: Formation_Water_Salinity, Parameter value: 20000
Parameter name: initial_pressure, Parameter value: 3500
Parameter name: current_pressure, Parameter value: 2500
```

Data Structures Comprehension

1 - List Comprehension

```
#Output = [expression for item in iterable if condition]

#expression: transformation / equation / output value
#item : current item from the existing list
#iterable: list
#if condition (Optional): filter decides which items to be included

wavelet = [0.2, 0.5, 0.8, 1.0, 0.7]
reflection_coeff = [0.1, 0.3, 0.6, 0.9, 0.5]
noise = [0.05, -0.02, 0.01, -0.03, 0.02]

Seismic_Response = []

for i in range(len(wavelet)):
    Seismic_value = reflection_coeff[i] * wavelet[i] + noise[i]
    Seismic_Response.append(Seismic_value)

print(Seismic_Response)
```

```
[0.07, 0.13, 0.49, 0.87, 0.37]
```

```
Seismic_Response = []
def Seismic_Response_Calc(reflection_coeff, wavelet, noise):
    for i in range(len(wavelet)):
        Seismic_value = reflection_coeff[i] * wavelet[i] + noise[i]
        Seismic_Response.append(Seismic_value)
    return Seismic_Response

Seismic_Response_Calc(reflection_coeff, wavelet, noise)
```

```
[0.07, 0.13, 0.49, 0.87, 0.37]
```

```
Seismic_Response = [reflection_coeff[i] * wavelet[i] + noise[i] for i
in range(len(wavelet))]

print(Seismic_Response)
```

```
[0.07, 0.13, 0.49, 0.87, 0.37]
```

```
def Seis_calc(reflection_coeff, wavelet, noise):
    Seismic_Response = [reflection_coeff[i] * wavelet[i] + noise[i]
for i in range(len(wavelet))]
    return Seismic_Response

Seis_calc(reflection_coeff, wavelet, noise)
```

```
[0.07, 0.13, 0.49, 0.87, 0.37]
```

```
wavelet = [0.2, -0.5, 0.8, -1.0, -0.7, -0.6, -0.3, -0.9, 0.4, 0.2,
           0.6, -0.4, 0.8, -0.9, -0.5, 0.7, -0.3, 0.7, -1.0, -0.5]

reflection_coeff = [0.5, 0.3, 0.6, -0.9, 0.5, -0.4, 0.8, -0.7, -0.3,
0.8,
                    0.4, -0.6, 0.6, 0.7, 0.2, 0.5, 0.9, 0.6, -0.3,
0.4]

noise = [0.05, -0.02, -0.01, 0.03, -0.02, 0.04, 0.01, -0.06, 0.05, -
0.03,
         0.04, 0.02, 0.06, -0.07, 0.03, 0.01, -0.05, 0.02, -0.04,
0.01]

#seismic Response
Seismic_Response = [round(reflection_coeff[i] * wavelet[i] + noise[i],
4) for i in range(len(wavelet))]

print(Seismic_Response)
```

```
[0.15, -0.17, 0.47, 0.93, -0.37, 0.28, -0.23, 0.57, -0.07, 0.13, 0.28,
0.26, 0.54, -0.7, -0.07, 0.36, -0.32, 0.44, 0.26, -0.19]
```

```
Pos_amp = []

for i in Seismic_Response:
    if i >=0:
        Pos_amp.append(i)
    else:
        pass

print(Pos_amp)
```

```
[0.15, 0.47, 0.93, 0.28, 0.57, 0.13, 0.28, 0.26, 0.54, 0.36, 0.44,
0.26]
```

```
#Positive_amplitude
Positive_amplitude = [ amplitude for amplitude in Seismic_Response if
amplitude>=0]

print(Positive_amplitude)
```

```
[0.15, 0.47, 0.93, 0.28, 0.57, 0.13, 0.28, 0.26, 0.54, 0.36, 0.44,
0.26]
```

```
#Absolute Amplitude

Abs_amp = []

for i in Seismic_Response:
    if i >=0:
        Abs_amp.append(i)
    else:
        Abs_amp.append(i*-1)

print(Abs_amp)
```

```
[0.15, 0.17, 0.47, 0.93, 0.37, 0.28, 0.23, 0.57, 0.07, 0.13, 0.28,
0.26, 0.54, 0.7, 0.07, 0.36, 0.32, 0.44, 0.26, 0.19]
```

```
Absolute_Amplitude = [amplitude for amplitude in Seismic_Response if
amplitude >=0 else amplitude*-1]
```

```
  Cell In[20], line 1
    Absolute_Amplitude = [amplitude for amplitude in Seismic_Response
if amplitude >=0 else amplitude*-1]

^
SyntaxError: invalid syntax
```

```
Absolute_Amplitude = [amplitude if amplitude >=0 else (amplitude*-1)
for amplitude in Seismic_Response ]

print(Absolute_Amplitude)
```

```
[0.15, 0.17, 0.47, 0.93, 0.37, 0.28, 0.23, 0.57, 0.07, 0.13, 0.28,
0.26, 0.54, 0.7, 0.07, 0.36, 0.32, 0.44, 0.26, 0.19]
```

2 - Dictionary Comprehension

```
#Output = { Key: Value expression for item in iterable if condition }

Seismic_Response = [0.15, -0.17, 0.47, 0.93, -0.37, 0.28, -0.23, 0.57,
-0.07, 0.13, 0.28, 0.26, 0.54, -0.7, -0.07, 0.36, -0.32, 0.44, 0.26, -
0.19]
```

```
high_amplitude = [ ]
low_amp = []


for amp in Seismic_Response:
    if amp >0.28 or amp <-0.28:
        high_amplitude.append(amp)
    elif -0.28 < amp <= 0.28:
        low_amp.append(amp)

print(high_amplitude)
print(low_amp)
```

```
[0.47, 0.93, -0.37, 0.57, 0.54, -0.7, 0.36, -0.32, 0.44]
[0.15, -0.17, 0.28, -0.23, -0.07, 0.13, 0.28, 0.26, -0.07, 0.26, -
0.19]
```

```
high_amplitude = [ amplitude for amplitude in Seismic_Response if
amplitude > 0.28 or amplitude < -0.28]
high_amplitude
```

```
[0.47, 0.93, -0.37, 0.57, 0.54, -0.7, 0.36, -0.32, 0.44]
```

```
high_amp_dict = {}
low_amp_dict = {}

for amp in Seismic_Response:
    if amp >0.28 or amp <-0.28:
        high_amp_dict[amp] = "High amplitude"
    elif -0.28 < amp <= 0.28:
        low_amp_dict[amp] = "Low Amplitude"

high_amp_dict
```

```
{0.47: 'High amplitude',
 0.93: 'High amplitude',
 -0.37: 'High amplitude',
 0.57: 'High amplitude',
 0.54: 'High amplitude',
 -0.7: 'High amplitude',
 0.36: 'High amplitude',
 -0.32: 'High amplitude',
 0.44: 'High amplitude'}
```

```
low_amp_dict
```

```
{0.15: 'Low Amplitude',
 -0.17: 'Low Amplitude',
 0.28: 'Low Amplitude',
 -0.23: 'Low Amplitude',
 -0.07: 'Low Amplitude',
```

```
 0.13: 'Low Amplitude',
 0.26: 'Low Amplitude',
 -0.19: 'Low Amplitude'}
```

```
high_amp_dict = { amplitude : "High Amplitude " for amplitude in
Seismic_Response if -0.28 < amplitude <= 0.28 }
```

```
high_amp_dict
```

```
{0.47: 'High Amplitude ',
 0.93: 'High Amplitude ',
 -0.37: 'High Amplitude ',
 0.57: 'High Amplitude ',
 0.54: 'High Amplitude ',
 -0.7: 'High Amplitude ',
 0.36: 'High Amplitude ',
 -0.32: 'High Amplitude ',
 0.44: 'High Amplitude '}
```

```
low_amp_dict = { amplitude : "low Amplitude " for amplitude in
Seismic_Response if -0.28 < amplitude <= 0.28 }
```

```
low_amp_dict
```

```
{0.15: 'low Amplitude ',
 -0.17: 'low Amplitude ',
 0.28: 'low Amplitude ',
 -0.23: 'low Amplitude ',
 -0.07: 'low Amplitude ',
 0.13: 'low Amplitude ',
 0.26: 'low Amplitude ',
 -0.19: 'low Amplitude '}
```

3 - Set Comprehension

```
#Output = {expression for item in iterable if condition }
```

Functions, Parameters, and Arguments

def fun(par1, par2): statements

fun(arg1, arg2)

```
def Res(phi, k, h):
    print(phi)
    print(k)
    print(h)

Res(0.18, 200, 50)
```

```
0.18
200
50
```

```
Res(0.18, 200)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 1
----> 1 Res(0.18, 200)

TypeError: Res() missing 1 required positional argument: 'h'
```

```
#Star operator   *params     *args


def Res(*params):
    print(f" Reservoir parameters are {params}")
    print(type(params))
```

```
Res(0.2)
```

```
 Reservoir parameters are (0.2,)
<class 'tuple'>
```

```
Res(0.2, 200, 120)
```

```
 Reservoir parameters are (0.2, 200, 120)
```

```
Res(0.2, 120)
```

```
 Reservoir parameters are (0.2, 120)
```

```
Res(0.2)
```

```
 Reservoir parameters are (0.2,)
```

```
def Res(*params):
    print(f" porosity {params[0]}")
    print(f" perm {params[1]}")
    print(f" Thickness {params[2]}")
```

```
Res(0.2, 200, 120)
```

```
 porosity 0.2
 perm 200
 Thickness 120
```

```
def total_thickness(*layers):
    thickness = 0
    for layer in layers:
```

```
        thickness+=layer
    print(f"Total thickness is {thickness} meters")
```

```
total_thickness(220, 120, 500, 340, 520)
```

```
Total thickness is 1700 meters
```

```
total_thickness(120, 500)
```

```
Total thickness is 620 meters
```

```
total_thickness(200, 500, "Three hundred")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[53], line 1
----> 1 total_thickness(200, 500, "Three hundred")

Cell In[50], line 4, in total_thickness(*layers)
      2 thickness = 0
      3 for layer in layers:
----> 4     thickness+=layer
      5 print(f"Total thickness is {thickness} meters")

TypeError: unsupported operand type(s) for +=: 'int' and 'str'
```

```
def Res(phi, k, h, *params):
    print(phi)
    print(k)
    print(h)
    print(params)
```

```
Res(0.18, 200, 100, 20, 0.7, 50)
```

```
0.18
200
100
(20, 0.7, 50)
```

```
Res(0.18, 200, 100)
```

```
0.18
200
100
()
```

```
Res(0.18, 200)
```

```
---------------------------------------------------------------------------
```

```
TypeError                                 Traceback (most recent call
last)
Cell In[57], line 1
----> 1 Res(0.18, 200)

TypeError: Res() missing 1 required positional argument: 'h'
```

```python
def Res(phi, k, *params, h):
    print(phi)
    print(k)
    print(h)
    print(params)


Res(0.18, 200, 0.2, 100, 100, 120)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[61], line 8
      4     print(h)
      5     print(params)
----> 8 Res(0.18, 200, 0.2, 100, 100, 120)

TypeError: Res() missing 1 required keyword-only argument: 'h'
```

```python
def Hydrocarbon_pore_volume(well_name, area, net_thickness, porosity,
*saturation_values):
    for sw in saturation_values:
        hcpv = area*net_thickness*porosity*(1-sw)

        print(well_name)
        print(f"for {sw} water saturation : HCPV is {hcpv:.2f} cubic
meters")

Hydrocarbon_pore_volume("well-x", 2000, 50, 0.25,0.2, 0.25, 0.3)
```

```
well-x
for 0.2 water saturation : HCPV is 20000.00 cubic meters
well-x
for 0.25 water saturation : HCPV is 18750.00 cubic meters
well-x
for 0.3 water saturation : HCPV is 17500.00 cubic meters
```

```python
def Hydrocarbon_pore_volume(well_name, area, net_thickness, porosity,
*saturation_values):
    for sw in saturation_values:
        for i in range(len(porosity)):
            hcpv = area*net_thickness*porosity[i]*(1-sw)
```

```
            print(well_name)
            print(f"for {sw} water saturation for {porosity[i]} : HCPV
is {hcpv:.2f} cubic meters")

Hydrocarbon_pore_volume("well-x", 2000, 50, [0.25, 0.28, 0.35],0.2,
0.25, 0.3)
```

```
well-x
for 0.2 water saturation for 0.25 : HCPV is 20000.00 cubic meters
well-x
for 0.2 water saturation for 0.28 : HCPV is 22400.00 cubic meters
well-x
for 0.2 water saturation for 0.35 : HCPV is 28000.00 cubic meters
well-x
for 0.25 water saturation for 0.25 : HCPV is 18750.00 cubic meters
well-x
for 0.25 water saturation for 0.28 : HCPV is 21000.00 cubic meters
well-x
for 0.25 water saturation for 0.35 : HCPV is 26250.00 cubic meters
well-x
for 0.3 water saturation for 0.25 : HCPV is 17500.00 cubic meters
well-x
for 0.3 water saturation for 0.28 : HCPV is 19600.00 cubic meters
well-x
for 0.3 water saturation for 0.35 : HCPV is 24500.00 cubic meters
```

```
def Hydrocarbon_pore_volume(well_name, area, net_thickness, porosity,
*saturation_values):
    for sw in saturation_values:
        for i in range(len(porosity)):
            hcpv = area*net_thickness*porosity[i]*(1-sw)

            print(well_name)
            print(f"for {sw} water saturation for {porosity[i]} : HCPV
is {hcpv:.2f} cubic meters")

Hydrocarbon_pore_volume("well-x", 2000, 50, [0.25, 0.28, 0.35], *[0.2,
0.25, 0.3])
```

```
well-x
for 0.2 water saturation for 0.25 : HCPV is 20000.00 cubic meters
well-x
for 0.2 water saturation for 0.28 : HCPV is 22400.00 cubic meters
well-x
for 0.2 water saturation for 0.35 : HCPV is 28000.00 cubic meters
well-x
for 0.25 water saturation for 0.25 : HCPV is 18750.00 cubic meters
well-x
for 0.25 water saturation for 0.28 : HCPV is 21000.00 cubic meters
well-x
```

```
for 0.25 water saturation for 0.35 : HCPV is 26250.00 cubic meters
well-x
for 0.3 water saturation for 0.25 : HCPV is 17500.00 cubic meters
well-x
for 0.3 water saturation for 0.28 : HCPV is 19600.00 cubic meters
well-x
for 0.3 water saturation for 0.35 : HCPV is 24500.00 cubic meters
```